

Fine-Tuning and LoRA

When prompting isn't enough · video 2:37

This lesson answers one question: when should you change the model itself, and how can you do it cheaply? **Fine-tuning** keeps training a model on examples of the behaviour you want. On the tiny GPT, full fine-tuning learns a new style but **forgets** the old one and produces a whole new model. **LoRA** freezes every original weight and trains two thin matrices per layer: **4% of the parameters**, almost the same result, and an adapter you can switch off or swap.

How to use this guide: master one concept before moving to the next.

1. Watch the whole video once, without stopping.
2. Then take the concepts **in order**. For each one: read the short explanation, look at the picture from the video (▶ is the time in the video), and answer every question in its *Check* without looking anything up.
3. Check your answers in the **Answers** section at the end (page 6).
4. **Pass mark: every question in the check correct.** If you missed one, rewatch that concept's video segment, reread the explanation, and redo only the questions you missed. Tick the concept off below, then move on.

It is strongly suggested that you **master each concept before starting the next one**. Each concept builds on the previous ones.

Concepts in this lesson

CONCEPT	VIDEO SEGMENT	QUESTIONS	MASTERED
1. Prompting, RAG or fine-tuning?	▶ 0:08-0:38	2	☐
2. Full fine-tuning: learns, and forgets	▶ 0:38-1:12	2	☐
3. LoRA in one picture	▶ 1:12-1:51	3	☐
4. Adapters you can switch and swap	▶ 1:51-2:37	2	☐

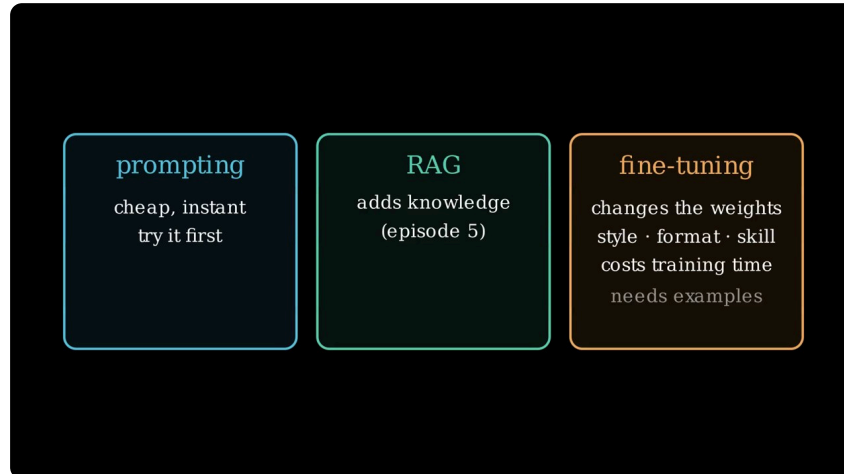
Before you start: How LLMs Work episodes 11-12 (training and the tiny GPT) and Foundations F03 (matrices). Code: `code/p09_lora` (CPU only; the first run trains the base model in about 25 minutes).

CONCEPT 1 OF 4

Prompting, RAG or fine-tuning?

▶ Video 0:08-0:38

Prompts, documents and tool results all change what goes **into** the model; the model itself never changes. **Fine-tuning** changes the **weights**, for when you want different behaviour every time without saying so in every prompt.



▶ 0:37 Prompting is cheap and instant; RAG adds knowledge; fine-tuning changes the weights.

Prompting is cheap and instant, and is the first thing to try. **RAG** adds knowledge. **Fine-tuning** is for a style, a format or a skill that is hard to describe in words; it costs training time, and you need examples.

KEY IDEA

Change the input first; change the model only when the behaviour cannot be prompted or retrieved.

Check 1: Prompting, RAG or fine-tuning?

Answer all 2 questions before looking at the answers. Pass mark: all correct.

1.1 MULTIPLE CHOICE

Your assistant must know this week's prices, which change every Monday. Best tool?

- A. Fine-tuning every week
- B. RAG over the current price list
- C. A higher temperature
- D. Training a new model

1.2 MULTIPLE CHOICE

You need every reply in a strict house style that is hard to describe, and you have 2,000 example replies. Best tool?

- A. Prompting alone
- B. RAG
- C. Fine-tuning (e.g. LoRA)
- D. Sampling at temperature 0

CONCEPT 2 OF 4

Full fine-tuning: learns, and forgets

▶ Video 0:38–1:12

The experiment uses the tiny GPT from How LLMs Work, trained on Shakespeare. Asked for a recipe, it writes Shakespeare; on the made-up recipes its loss is **2.77**.

trained on Shakespeare

tiny GPT
818,241 parameters
How LLMs Work, ep. 12

RECIPE:
Go'er her man lord, let seen is eath
And him be made your stormer and what have shing of
'He would gave you and frait in their that as

loss on made-up recipes: 2.77

▶ 0:50 The Shakespeare-trained tiny GPT, asked for a recipe, writes Shakespeare (recipe loss 2.77).

option 1 · full fine-tuning

all 818,241 weights change

RECIPES: LEMON AND LEEK PASTA
Ingredients:
- three handfuls of mushroom
- one cup of apple
- half a cup of milk
Steps:
First, roast the plum ...

recipes, before	2.77
recipes, after	0.25
Shakespeare, before	1.60
Shakespeare, after	4.40 it forgot

+ a whole new copy of the model

▶ 1:11 Full fine-tuning: recipe loss 0.25, but Shakespeare loss 1.60 → 4.40.

Full fine-tuning: keep training on 600 steps of recipes, updating all **818,241** weights. Recipe loss drops to **0.25** and it writes a perfect recipe. But its Shakespeare loss jumps from **1.60 to 4.40**: it **forgot** (this is called catastrophic forgetting). And you now have a whole new copy of the model.

KEY IDEA

Updating every weight for a new task can overwrite what the model knew, and every task needs its own full copy.

Check 2: Full fine-tuning: learns, and forgets

Answer all 2 questions before looking at the answers. Pass mark: all correct.

2.1 CALCULATION

By how much did the Shakespeare loss rise after full fine-tuning?

2.2 TRUE OR FALSE

"After full fine-tuning on recipes, the model is better at both recipes and Shakespeare."

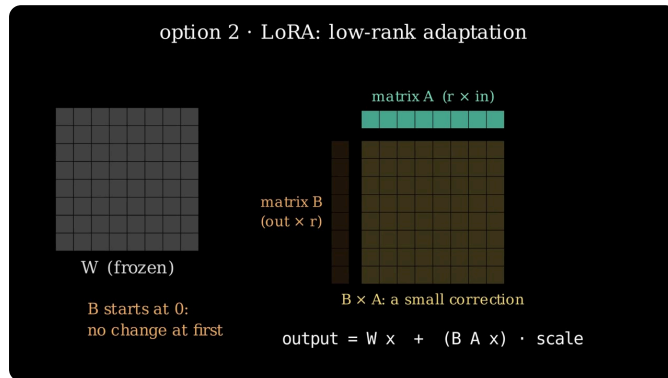
True ☐ False ☐

CONCEPT 3 OF 4

LoRA in one picture

▶ Video 1:12-1:51

LoRA (low-rank adaptation): freeze every original weight. Next to each weight matrix W , add two thin matrices, A ($r \times \text{in}$) and B ($\text{out} \times r$). Their product $B \times A$ is a small correction added to the frozen output: $\text{output} = W \times x + (B \times A \times x) \cdot \text{scale}$. B starts at zero, so at first nothing changes. Only A and B are trained.



▶ 1:32 LoRA: W stays frozen; $B \times A$, two thin matrices, adds a small correction.



▶ 1:49 32,768 trainable numbers (4%), a 128 KB adapter, and almost the same recipe loss.

With rank $r = 4$ on every attention and MLP layer, that is **32,768** trainable numbers, **4.0%** of the model; the adapter is a **128 KB** file (the model is 3,196 KB). Recipe loss: **0.27**, almost the same as full fine-tuning (0.25).

KEY IDEA

Many useful changes are low-rank: two thin matrices can express them with a small fraction of the parameters.

Check 3: LoRA in one picture

Answer all 3 questions before looking at the answers. Pass mark: all correct.

3.1 CALCULATION

A frozen layer maps 128 inputs to 384 outputs (the attention's qkv). How many trainable numbers do A and B add with rank $r = 4$?

3.2 SHORT ANSWER

Why does B start at zero?

3.3 CALCULATION

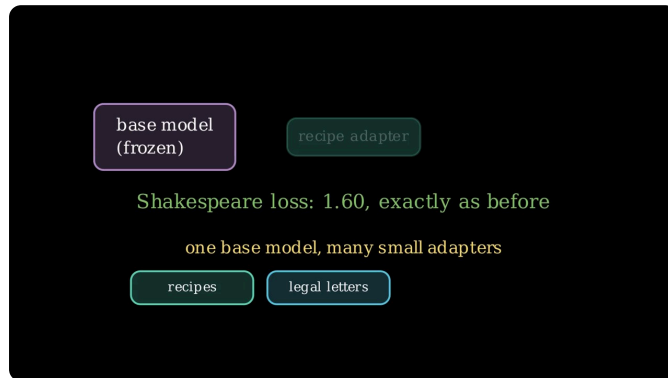
A full $128 \rightarrow 384$ weight matrix has how many numbers, and what fraction of that is the rank-4 LoRA pair?

CONCEPT 4 OF 4

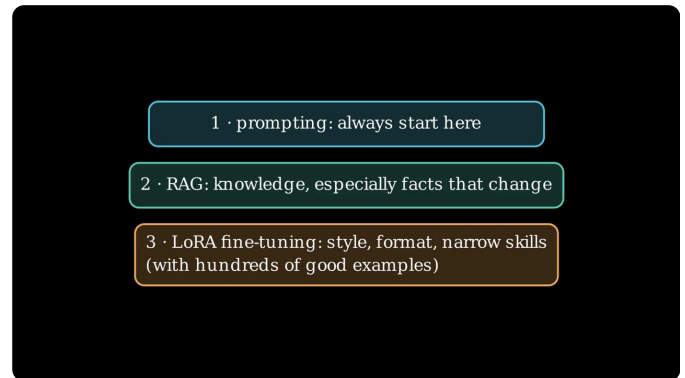
Adapters you can switch and swap

▶ Video 1:51-2:37

Switch the adapter off, and the Shakespeare loss is back to exactly **1.60**: the original weights never changed. Keep one base model and **swap** small adapters: one for recipes, one for legal letters, one per customer. In code, LoRA is a few lines: wrap a linear layer, freeze it, add A (random) and B (zero); the output is the frozen layer plus x times A, times B, times a scale.



▶ 2:05 Adapter off: Shakespeare loss is exactly 1.60 again. One base model, many small adapters.



▶ 2:31 When to use what.

So: start with **prompting**. Add **RAG** for knowledge, especially facts that change. **Fine-tune with LoRA** for style, format and narrow skills, when you have hundreds of good examples.

KEY IDEA

A LoRA adapter is a small, removable add-on: the base model stays intact and shareable.

Check 4: Adapters you can switch and swap

Answer all 2 questions before looking at the answers. Pass mark: all correct.

4.1 TRUE OR FALSE

“With the LoRA adapter switched on, the Shakespeare loss is also 1.60.”

True ☐ False ☐

4.2 HANDS-ON CODE

Try it yourself. In `code/p09_lora/lora.py`, change `add_lora(tuned, r=4)` to `r=1` and run it. (a) How many trainable parameters, and what recipe loss? (b) Compare a sample with rank 4's.

```
adapters = add_lora(tuned, r=1)
```

ANSWERS

Answers and explanations

Mark each answer right or wrong. If a concept has any wrong answer, rewatch its segment (▶) and redo the questions you missed before moving on.

1. Prompting, RAG or fine-tuning? · rewatch ▶ 0:08-0:38

- 1.1 B. Changing facts belong in the library, not in the weights.
- 1.2 C. A consistent style learned from many examples is what fine-tuning is for.

2. Full fine-tuning: learns, and forgets · rewatch ▶ 0:38-1:12

- 2.1 2.80. $4.40 - 1.60 = 2.80$.
- 2.2 False. Recipes improved ($2.77 \rightarrow 0.25$) but Shakespeare got much worse ($1.60 \rightarrow 4.40$).

3. LoRA in one picture · rewatch ▶ 1:12-1:51

- 3.1 2,048. A is $4 \times 128 = 512$, B is $384 \times 4 = 1,536$; $512 + 1,536 = 2,048$.
- 3.2 So that $B \times A$ is zero at the start: the model behaves exactly like the base model until training begins. The code confirms it: before training, the recipe loss is 2.77, identical to the base model.
- 3.3 49,152; about 4.2%. $128 \times 384 = 49,152$; $2,048 / 49,152 \approx 0.042$.

4. Adapters you can switch and swap · rewatch ▶ 1:51-2:37

- 4.1 False. With the adapter on it is 3.92: the adapter steers the model toward recipes. Only switched off is it exactly 1.60.
- 4.2 (a) 8,192 trainable parameters (1.0%, a 32 KB adapter); recipe loss 0.39 (rank 4: 0.27). (b) The format is learned but sloppier, e.g. "RECIPE: GERO AND TAR: ELY AND PLUQ STEW ... half a minut or a warm pan." Rank sets the adapter's capacity: rank 1 is cheaper but cannot capture the change as well. (Exact samples depend on the random state; the $r = 1$ numbers come from a run without the full fine-tuning step before it.)